

Article

A Method for Complex Question-Answering over Knowledge Graph

Lei Yang ¹, Haonan Guo ¹, Yu Dai ^{2,*} and Wanheng Chen ¹

¹ Key Laboratory of Intelligent Computing in Medical Image of Ministry of Education, College of Computer Science and Engineering, Northeastern University, Shenyang 110169, China; yanglei@mail.neu.edu.cn (L.Y.); 2101722@stu.neu.edu.cn (H.G.)

² College of Software, Northeastern University, Shenyang 110169, China

* Correspondence: daiy@swc.neu.edu.cn

Abstract: Knowledge Graph Question-Answering (KGQA) has gained popularity as an effective approach for information retrieval systems. However, answering complex questions involving multiple topic entities and multi-hop relations presents a significant challenge for model training. Moreover, existing KGQA models face difficulties in extracting constraint information from complex questions, leading to reduced accuracy. To overcome these challenges, we propose a three-part pipelined framework comprising question decomposition, constraint extraction, and question reasoning. Our approach employs a novel question decomposition model that uses dual encoders and attention mechanisms to enhance question representation. We define temporal, spatial, and numerical constraint types and propose a constraint extraction model to mitigate the impact of constraint interference on downstream question reasoning. The question reasoning model uses beam search to reduce computational effort and enhance exploration, facilitating the identification of the optimal path. Experimental results on the ComplexWebQuestions dataset demonstrate the efficacy of our proposed model, achieving an *F1* score of 72.0% and highlighting the effectiveness of our approach in decomposing complex questions into simple subsets and improving the accuracy of question reasoning.

Keywords: knowledge graph; question-answering over knowledge graph; complex question; deep neural networks



Citation: Yang, L.; Guo, H.; Dai, Y.; Chen, W. A Method for Complex Question-Answering Over Knowledge Graph. *Appl. Sci.* **2023**, *13*, 5055. <https://doi.org/10.3390/app13085055>

Academic Editor: Juan Francisco De Paz Santana

Received: 30 March 2023

Revised: 11 April 2023

Accepted: 13 April 2023

Published: 18 April 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

With the rise of large-scale knowledge graphs [1], such as Wikidata [2], DBpedia [3], Freebase [4], etc., users are finding it difficult to quickly obtain a large amount of valuable knowledge from them due to their large volume and complex data structure. Knowledge graph question-answering (KGQA) has emerged as a critical tool for addressing this challenge and has garnered considerable attention. When compared to traditional question-answering tasks, KGQA improves semantic understanding and answer accuracy [5]. As shown in Table 1, in this paper: (1) Simple questions containing one topic entity with one relation [6,7] and multi-hop questions containing one topic entity with multiple relations [8] are collectively referred to as simple questions. (2) Aggregate complex questions [9] containing multiple topic entities with multiple relations are referred to as complex questions. Many studies have been published by researchers to improve simple question answering over knowledge graph accuracy. Existing studies have achieved an accuracy rate of up to 78.1% [10] on SimpleQuestions [11], a dataset of simple questions representing single-hop relations. However, due to the complexity of entities and relations, there is still a lot of research space and application value for complex questions, and the main challenges we face are as follows.

1. Complex question has multi-topic entities and multi-hop relations, which present difficulties for neural networks to learn directly.

2. Complex question components contain various constraint information such as time, space, numerical operations, etc., which can interfere with the downstream reasoning model.
3. The knowledge graph is large and the question may not contain all path information, the task of identifying a graph's locating number is a non-deterministic polynomial-time hard problem (NP) [12]. However, the model needs to quickly identify the optimal path from it and make reasonable inferences about the missing path information.

Table 1. Example questions for different types.

Question Type	Subtype	Example
Simple	1 entity, 1 relation	Who is the author of the Lord of the Rings?
	1 entity, multi-relations	Where is the birthplace of the author of the Lord of the Rings?
Complex	multi-entities, multi-relations	Which rivers flow through both China and India?
		Which book was first co-authored by Marx and Engels?

To summarize, we design a solution for complex question-answering tasks over knowledge graphs and decompose them into three tasks by simulating how humans solve complex questions, question decomposition, constraint extraction, and question reasoning. In the question decomposition section, we develop a model called BTAM to address the problem that complex questions are difficult to train directly in neural networks by using a pre-trained model as an encoder to encode the semantic information of the question and designing a dependency encoder to encode the syntactic structure between words in the question. The model can better represent complex questions. In the constraint extraction section, we propose a constraint representations method and design a model that, while extracting constraint information also determines the subject on which the constraint information acts via dependency analysis, potentially increasing the question reasoning accuracy rate. In the question reasoning section, we present the model HBHR, which uses beam search for iterative reasoning in the knowledge graph based on the information derived from the first two sections, and can improve the accuracy rate while reducing the search space, to solve the problem whereby the optimal path cannot be found quickly due to the excessively large size of the knowledge graph and the missing path information cannot make a reasonable inference. Furthermore, for similarity detection, the model employs a pseudo-siamese network, which engages different encoders to encode the question and paths for computation, and the encoders are designed in a learnable form to obtain a better vector representation during training, thereby improving the reasoning process interpretability and missing path information exploration.

The question-answering model for complex questions proposed in this paper uses a pipeline structure with interpretability and a neural network model for information extraction, which reduces data quality requirements and renders the model more parsable, accurate, and robust. The main contributions of this paper are as follows.

- A question decomposition model based on pre-training model and dependency analysis is created, which can break down a complex question into a set of several simple questions for subsequent model processing. This solves the issue that complex questions have multi-topic entities and multi-hop relations, which make it difficult for neural networks to learn directly.
- A constraint representation method that includes temporal, spatial, and numerical operations is established to address the issue that the question constraints can conflict with downstream reasoning models, and an extraction model for separating the constraint information in the question is designed to increase the accuracy rate of the question reasoning.

- A question reasoning model based on beam search is intended to address the problem that the optimal path is challenging to find quickly due to the size of the knowledge graph, which can improve the model's accuracy while narrowing the search space.

The rest of this paper is organized as follows: Section 2 presents the related work on KGQA tasks. We first describe the general structure of the model in Section 3 then introduce the three sub-models for question decomposition, constraint extraction, and question reasoning. The experimental Section 4 presents numerous experimental findings and ablation studies are conducted to support the model's validity. The conclusions and accompanying prospects are presented in Section 5.

2. Related Work

In this section, we present the work related to the study of this paper, including the semantic parsing-based KGQA task, the question decomposition task, the constraint extraction task, and the question reasoning task.

2.1. The Semantic Parsing Approaches

Semantic parsing-based question-answering systems typically convert the natural language-processing-generated question into some intermediate logical form, such as a query graph or query tree, and then convert the intermediate logical form into query statements, such as SPARQL [13], GraphQL [14], etc., and finally send the query statements to an executor for execution. STAGG [6] converts the query graph's constraint nodes into SPARQL for execution and then adds the constraint nodes to the primary relational chain to acquire the answer framework. Multi-constraint query graph generation (MultiCG) [15] based on the STAGG framework was developed in response to criticism espousing that SATGG cannot handle some complex constraints. MultiCG extends the types of constraints and types of operations and develops more rules to parse complex problems. A query graph generation strategy was put forth that gradually condenses the entire knowledge graph to the desired query pattern [16].

Deep learning models were also incorporated into the query graph generation process. Since poor entity linking results directly impact the performance of the question-answering system, HR-BiLSTM [7] improves the entity linking and relation detection approaches to improve accuracy. To determine the degree to which query graphs and questions are similar to one another, a similarity computation model based on STAGG was proposed [17]. In order to determine which query graph is the correct response, the model builds a neural network using BiGRU and ranks them according to how similar they are. Additionally, a slot matching model based on a self-attention mechanism showed that performance is greatly enhanced when the model is applied to small QA datasets after learning experiences on large QA datasets [18].

In addition to the aforementioned studies, other researchers have parsed the issue using various structures. The tree structure serves as a query graph, the question is parsed into a tree structure, and the tree is then encoded into a sequence for subsequent operations by using a tree-based LSTM [19]. The complex question is decomposed into a number of simple questions, and the constraints in the question are abstracted, resulting in a computational tree generated by merging a series of simple questions and constraints [20]. Moreover, the operations are redefined to build the query graph, and the model chooses the top rated one in the query graph set as the answer output for the following session, with overall training accomplished using the policy gradient in reinforcement learning [9].

This approach, however, has the disadvantages of difficult segmentation training, a reduced ability to parse complex questions, insufficient question representation, and a high cost.

2.2. Question Decomposition Approaches

There has been little research on the use of question decomposition approaches to understand complex questions. Earlier research used a rule-based approach to parallel and nested decomposition of questions using hand-customized rules based on word semantics

to achieve question decomposition [21]. Following that, SPLITQA [20] decomposes complex questions into groups of simple questions and reflects complex question semantics by mixing simple questions. A multi-hop question can be broken down into a series of single-hop questions using DECOMPRC [22], which can then combine the single-hop replies to produce the desired result. There is also the application of reinforcement learning for question decomposition, which allows an agent to learn strategies for decomposing complex questions into simple ones while determining the relations between simple questions utilizing three independent relational components [23]. Other approaches use unsupervised seq2seq learning to build a pseudo-decomposition model that divides a question into several small questions and then builds a recombination model that combines the answers to these simulated questions to achieve the final answer [24]. Unsupervised processes have been utilized to construct decompositions by mapping a complex question to a collection of feasible subproblems in a corpus of simple questions [25]. Decomposition can be used to investigate the model and produce explanations for its logic, according to a recent use case [26].

However, the analysis above shows the prior models that employed the question decomposition approach to comprehend complex questions using only a small amount of data for training, and it is challenging to obtain good question representations when the traditional models encode only the semantic aspects in the absence of contextual information of the question.

2.3. Question Constraint Extracting Approaches

Semantic parsing-based systems are more prevalent in the study of question constraints. For instance, STAGG [6] displays the constraint information in a question using aggregation functions. The staged query graph generation technique generates interpolation graphs in five stages [17]. Furthermore, SubGTR [27] with time-constrained reasoning uses the background knowledge stored in the temporal knowledge graph to reformulate the question to obtain explicit temporal constraints.

All of the above models, however, are only for a single type of constraint and cannot solve questions with multiple constraint types. Furthermore, the models are inefficient during training due to the large search space.

2.4. Question Reasoning Approaches

Multi-hop questions are those that contain a topic entity and multiple relations. IRN [8] uses a hop-by-hop inference to approach the response entity and selects the next hop path by evaluating the similarity between the input question and the current path. UHop [28] turns multi-hop relational extraction for complicated issues into multiple single-hop relational extraction plus termination decision. Other models, such as PullNet [29,30], use an iterative information retrieval technique to generate subgraphs from the knowledge graph and then use the generated subgraphs for answer prediction.

Another class of models based on semantic extraction, such as the joint relation representation method [31] and SGREADER [32], intercepts the entire knowledge graph or all entities and relations within the K-hop range centered on the topic entity into subgraphs, and maps both the question and the subgraphs in the knowledge graph.

The reinforcement-learning-based approach [33] provides a new perspective on knowledge reasoning that considerably improves its effectiveness and diversity in terms of interactive modeling ideas. Deeppath [34], as the archetypal approach, sees knowledge entities as state spaces that roam between entities via selection relations. Deeppath is awarded if it reaches the proper response entity. SRN [35] employs reinforcement learning for inference and handles multi-hop questions as sequential decision problems. Furthermore, a deep reinforcement-learning-based dynamic knowledge graph reasoning framework is proposed, which learns to browse the network under the constraint of input questions to locate likely target responses [36]. To obtain a dynamic reasoning model, the model is developed with a new reward function and dynamic rewards based on the proposed dynamic reasoning assumptions.

However, when the questions are directly supplied into the neural network for feature representation, the constraint information will certainly produce interference. Furthermore, because the knowledge graph is so large, existing models usually search via random walk or enumeration, which is inefficient.

3. Proposed Model

3.1. Model Overview

In this paper, we propose a new approach to solving complex questions by simulating the process of humans answering complex questions, based on an analysis of the composition of previous solutions and complex questions. As shown in Figure 1, we use a pipelined framework to break down the complex question-answering over knowledge graph task into three subtasks: complex question decomposition subtask, question constraint extraction subtask, and question reasoning subtask.

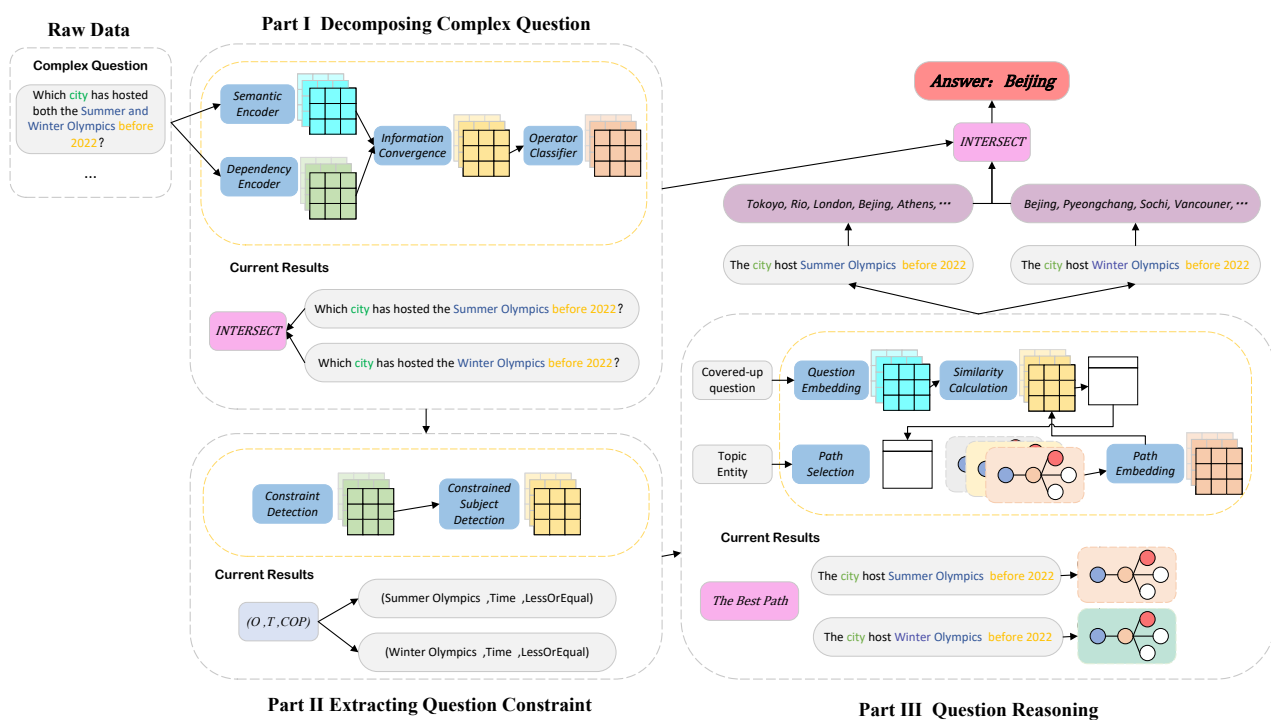


Figure 1. Overall model structure.

First, we propose BTAM, a complex question decomposition model that decomposes a complex question entered by the user into several simple questions and computes the logical operations between these simple questions. For semantic encoding in BTAM, the pre-trained model Bert is used, and the dependency encoder is used to capture the dependencies between words in a sentence. The outputs of both encoders are combined using the attention mechanism. Meanwhile, the model uses transfer learning to train. We then create a set of constraint expression paradigms that contain as much of the expression form of the centralized constraints as possible. Furthermore, we build a constraint extraction model that is utilized to automatically separate the constraint information in a simple question. Finally, we use the HBHR model to handle the constraints and key information in simple questions. Based on the input question, topic entity, and constraint information, the model deduces the path to the answer entity autonomously. Path selection is aided by beam search to improve path query efficiency, while similarity detection of question and path is accomplished by pseudo-siamese network to improve model performance.

3.2. Complex Question Decomposition

The BTAM sequence tagging model proposed in this paper is based on deep learning, and its overall structure is depicted in Figure 2. The model employs the Stanford Core Nlp tool [37] for question topic entity extraction. The sequence tagging section of the model is on the left and is responsible for tagging the question based on the topic entities to construct the relevant sequences. This section employs Bert as the semantic encoding information of words, designs the inter-word dependency encoder to encode the syntactic structure information of sentences, and utilizes the attention mechanism in the decoding stage to fuse information from two different types of encoders to achieve a better word representation vector. On the right side of the model is the operator classification section, which is responsible for choosing which operation logic is used to integrate the outcomes of the preceding decomposition. There are two sorts of logical operations in this work, namely $OP \in \{MERGE, INTERSECT\}$, which signifies the merge operation and the intersection operation, respectively.

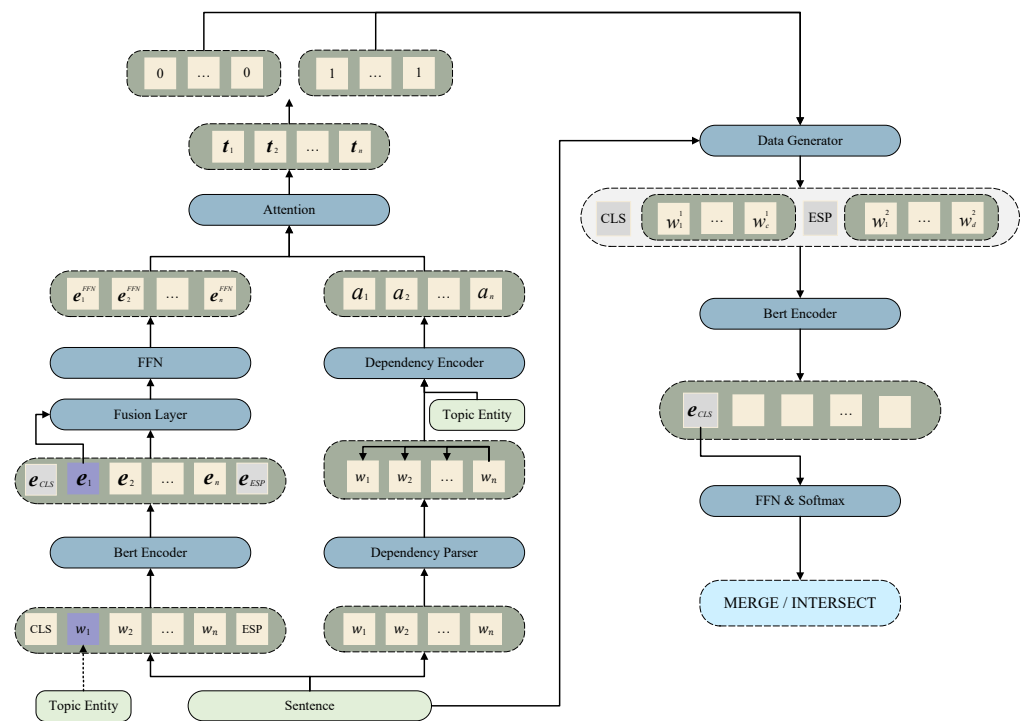


Figure 2. Complex question decomposition model.

The model's input is divided into two parts: sentence information and topic entity information. The sentence is denoted as $S = \{w_1, w_2, \dots, w_n\}$, where w_i denotes the i th word in the sentence and n denotes the number of words. S will be encoded in parallel by the semantic encoder and the dependency encoder. The topic entity in this sentence guides the semantic encoder and the dependency encoder in information selection.

3.2.1. Semantic Encoder

The first part of sequence tagging is the semantic encoder, which uses the pre-trained model Bert for semantic extraction. First, add CLS and ESP flag to S and generate position embedding $\mathbf{e}_i^p$, segment embedding $\mathbf{e}_i^s$, and token embedding $\mathbf{e}_i^t$ for the i th word, then submit these three embeddings to Bert. Finally, we obtain the context representation matrix of the sentence $\mathbf{E}_{bert} = [\mathbf{e}_{CLS}, \mathbf{e}_1, \dots, \mathbf{e}_n, \mathbf{e}_{ESP}]$, as shown in Equation (1):

$$\mathbf{e}_i = \text{Bert}(\mathbf{e}_i^p + \mathbf{e}_i^s + \mathbf{e}_i^t) \quad (1)$$

Although $\mathbf{E}_{bert}$ contains $\mathbf{e}_{CLS}$ and $\mathbf{e}_{ESP}$, they are not used in the subsequent step. $\mathbf{E}_{bert}$ is then sent to the fusion layer for information modification. The input to the fusion layer consists of two components: the topic entity vector $\mathbf{q}$ and the context representation matrix $\mathbf{E}_{bert}$. The fusion layer is designed to follow the attention mechanism and is in charge of information filtering of $\mathbf{E}_{bert}$ according to $\mathbf{q}$ to improve information aggregation and attenuate external noise. The above process is shown in Equations (2)–(4):

$$s(\mathbf{e}_i, \mathbf{q}) = \mathbf{v}^T \tanh(\mathbf{W}_e \mathbf{e}_i + \mathbf{U}_q \mathbf{q}) \quad (2)$$

$$\mathbf{ff}_i = \text{softmax}(s(\mathbf{E}_{bert}, \mathbf{q})) = \frac{\exp(s(\mathbf{e}_i, \mathbf{q}))}{\sum_{j=1}^n \exp(s(\mathbf{e}_j, \mathbf{q}))} \quad (3)$$

$$\mathbf{e}'_i = \mathbf{ff}_i \mathbf{e}_i \quad (4)$$

where $\mathbf{v}$, $\mathbf{W}_e$ and $\mathbf{U}_q$ are trainable weights. $\mathbf{e}'_i$ is the i th word vector obtained after fusion layer processing. The feed-forward network (FFN) layer receives the word vector matrix $\mathbf{E}_{att} = [\mathbf{e}'_1, \mathbf{e}'_2, \dots, \mathbf{e}'_n]$ for further processing. FFN is designed to be used here for two purposes: first, to improve the semantic encoder's network memory capacity; second, to decrease the dimensionality of the vector in order to fuse it with the dependency encoder's output. The process is represented by Equation (5):

$$\mathbf{E}_{FFN} = \max(0, \mathbf{E}_{att} \mathbf{W}_{att} + \mathbf{b}_{att}) \mathbf{W}'_{att} + \mathbf{b}'_{att} \quad (5)$$

where $\mathbf{E}_{FFN} = [\mathbf{e}_1^{FFN}, \mathbf{e}_2^{FFN}, \dots, \mathbf{e}_n^{FFN}]$ is the word vector matrix obtained after the FFN layer. $\mathbf{W}_{att}$, $\mathbf{W}'_{att}$, $\mathbf{b}_{att}$ and $\mathbf{b}'_{att}$ are trainable weights.

3.2.2. Dependency Encoder

The second part is the word dependency encoder. In contrast to the semantic encoder which needs to encode the word context representation vector, the dependency encoder needs to extract the dependency relations between words. Dependency parsing is a method of representing sentence structure based on word relations, which can connect the relations of distant constituents, and this type of relation across the order of words can reveal the dependency relations between words. The dependency-directed graph is commonly used to describe dependence relations, and the distance between nodes in the graph can reflect the strength of the dependency relations, which is what the dependency encoder needs to extract. Based on the dependency directed graph, the encoder generates $\mathbf{W}_{dep} \in \mathbb{R}^{n \times n}$ reflecting the strength of the dependency relation between words. Figure 3 shows an example of producing $\mathbf{W}_{dep}$ from the dependency graph, and the generation of $\mathbf{W}_{dep}$ follows the rules stated below.

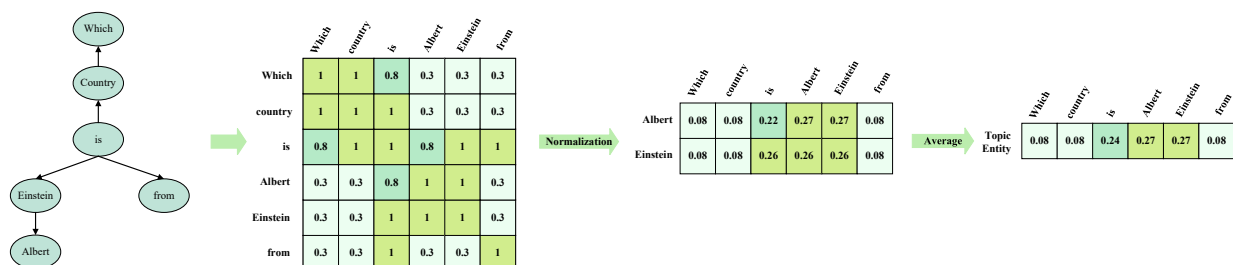


Figure 3. Dependency matrix generation example, where $\alpha = 0.8$ and $\varepsilon = 0.3$. For the sentence "Which country is Albert Einstein from", there is a weak dependency between "is" and "Albert" and a strong dependency between "is" and "from". After generating the dependency matrix, we need to select the vectors corresponding to this topic entity, normalize the vectors by rows, and then average them by columns.

1. The $d_{i,j}$ in the dependency matrix $\mathbf{W}_{dep}$ represents how strongly or weakly the j th word has an impact on the i th word in the dependency relation.
2. When the i th word travels through the relation only once to reach the j th word, it indicates that they have a strong dependency such that $d_{i,j} = 1$.
3. When word w_i takes two or more hops to reach word w_j , this indicates that they have weak dependency. At this point, $d_{i,j}$ will decay according to the shortest path between w_i and w_j based on the distance (number of hops), that is, $d_{i,j} = \alpha d_{i,j-1}$, where α is the decay coefficient and w_{j-1} is the nearest word to w_j on the shortest path between w_i and word w_j .
4. When w_i and w_j are not reachable, define $d_{i,j} = \varepsilon$ and ε denotes the minimum dependency.

Following the generation of $\mathbf{W}_{dep}$, the vector corresponding to the topic entity is chosen, and the vector is normalized by row first and then averaged by column. The goal is to create the weight vector for the topic entity.

The model employs the attention mechanism to merge the dependency information into the semantic information after the two encoders individually extract the semantic and dependency information. This procedure is described in Equation (6):

$$\mathbf{t}_i = a_i \mathbf{e}_i^{FFN} \quad (6)$$

where $\mathbf{t}_i$ is the vector of w_i after information fusion, a_i is w_i 's weight information. $\mathbf{T} = [\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_n]$ is then passed into the sequence tagger for question tagging, which first reduces $\mathbf{T} \in \mathbb{R}^{l \times n}$ to $\mathbf{T}' \in \mathbb{R}^{l \times 2}$ before determining its probability on 0 or 1, where 1 denotes the word in the decomposed simple question and 0 indicates the word is not part of the simple question. Equation (7) shows the process of obtaining $\mathbf{T}'$:

$$\mathbf{T}' = \mathbf{T} \mathbf{W}_F \quad (7)$$

where l is the length of the word vector, $\mathbf{W}_F \in \mathbb{R}^{n \times m}$ represents the trainable matrix, and m represents the number of classification. Here $m = 2$, which means it is a binary classification problem. For the complex question decomposition task, we model it as a sequence tagging task and use cross entropy as the loss function. The equation is shown in Equation (8):

$$Loss_1 = \frac{1}{N} \sum_i^n l_i = \frac{1}{N} \sum_i^n -[y_i \log p_i + (1 - y_i) \log(1 - p_i)] \quad (8)$$

where p_i is the probability that the model predicts w_i to be positive, that is, the probability of belonging to the simple question, and y_i is the actual label.

3.2.3. Operator Classifier

The input of this part is $S_{OP} = \{CLS, w_1^1, \dots, w_c^1, ESP, w_1^2, \dots, w_d^2\}$, consists of two parts and is split by the *ESP* identifier, where w_i^r means the i th word in the r th simple question. This model is in charge of deciphering which of the two parts is linked by the operational logic in $OP \in \{MERGE, INTERSECT\}$. S_{OP} is passed through Bert to obtain the representation matrix $\mathbf{E}_{OP}$. Only the vector $\mathbf{e}_{CLS}$, which contains global semantic information, is employed as the classification signal in $\mathbf{E}_{OP}$, which is then supplied into the subsequent model. FFN is used to lower the dimension of the classification signal, which is then normalized to forecast the classification outcome. For the operator classification task, which is essentially a binary classification task, a multiclassification cross entropy loss function is used in the training process, as shown in Equation (9):

$$Loss_2 = - \sum_{i=1}^m y_i \log p_i \quad (9)$$

where m is the number of classifications, y_i is the label, $y_i = 1$ if the classification is i otherwise $y_i = 0$; and p_i is the neural network output as the probability that the classification is i consisting of the softmax output.

3.3. Question Constraint Extraction

The goal of the constraint extraction subtask is to discover the constraints in the question and determine the kind of constraints and corresponding operands. The constraint extraction model proposed in this paper takes as input a given question $S_S = \{w_1, w_2, \dots, w_n\}$, and it detects the constraint information present in the question $C = \{c_1, c_2, \dots, c_n\}$, where $c_i = (O, T, COP)$, O is the constrained object, T is the constraint type, and COP is the set of constraint operands.

Figure 4 depicts the model. The constraint detection section of the model is on the left side and is responsible for recognizing T and COP from the question. It comprises of a semantic encoding layer, a semantic reasoning layer, and a constraint type detection layer. The constrained subject detection section of the model, which consists of a constraint subject detection layer and a fully connected layer, is on the right side of the model and is responsible for calculating O . In the following, we first give the range of values of T and COP , and then introduce each section of the model.

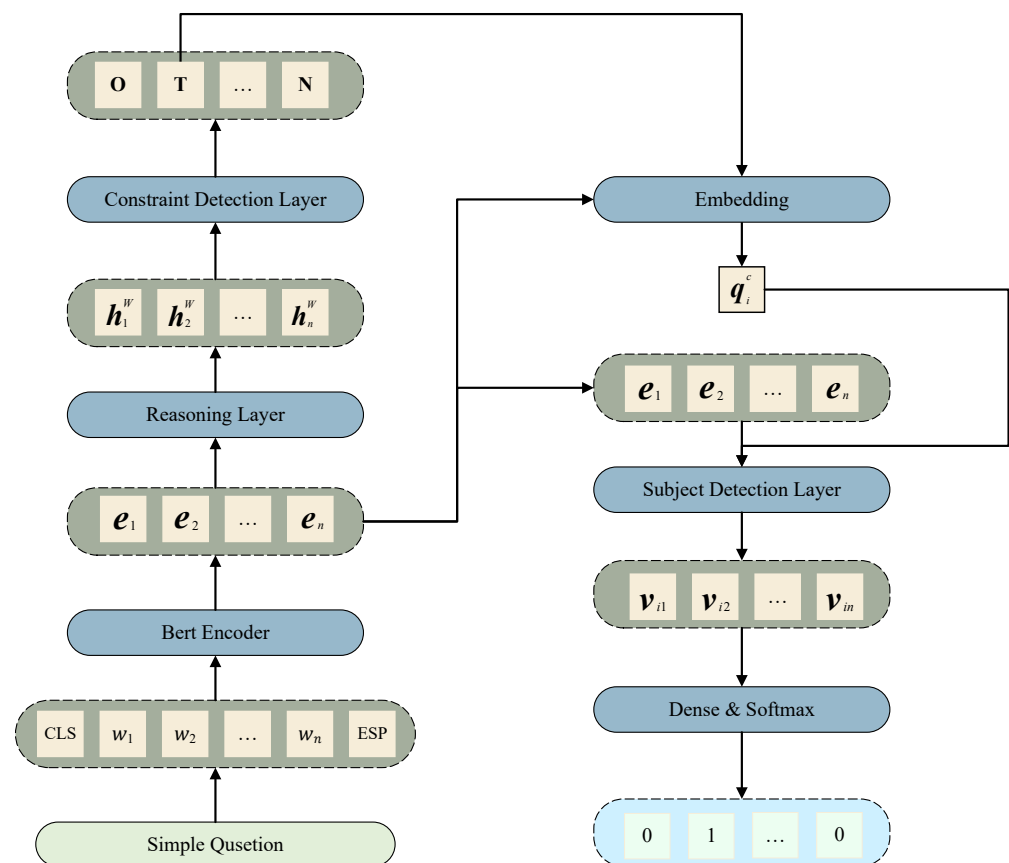


Figure 4. Question constraint extraction model.

3.3.1. Constraint Paradigm Definition

Constraints are mostly used in temporal, spatial, numerical operations and other terms. This section proposes several constraint paradigms to represent constraint information in a normative way by analyzing the representation of constraints in the dataset for use in the reasoning system that follows.

For temporal constraints, it was found through question analysis that they are usually divided into explicit temporal constraints and implicit temporal constraints. Most questions

include an implicit temporal constraint, which refers to the moment the question is posed, which is typically thought of as the present (now) and does not usually call for special consideration. Explicit temporal constraints are classified into two types: point-in-time constraints and time-period constraints. Point-in-time constraints are usually expressed in the question using time gerunds and contain obvious point-in-time information, whereas time-period constraints are usually expressed in the question using *between*, *during*, *after*, *before*, and other words followed by time information. We define three operators for time-period constraint: *Equal*, *LessOrEqual*, and *GreaterOrEqual*, each with two operands (A, B). $\langle A, \text{Equal}, B \rangle$ means that A and B are equal, $\langle A, \text{LessOrEqual}, B \rangle$ means that A is less than or equal to B , and $\langle A, \text{GreaterOrEqual}, B \rangle$ means that A is greater than or equal to B .

Spatial constraints are usually expressed by the adverbial of place, and the same triad represents a question with completely different answers depending on the spatial constraint. We define the operator *Equal* for spatial constraints, which has two operands (A, B), $\langle A, \text{Equal}, B \rangle$ indicates that A and B are equal.

For the numerical constraints, according to the question analysis, we find that they are usually expressed in terms of *min*, *max*, *top*, *first*, *second*, *biggest*, etc. Because the object of these constraint types is a set that must be sorted before the operation, two types of operators are defined: *Sort* and *Select*, where $\text{Sort}(A, \text{attr}, \text{order})$ is the definition of the *Sort* operator, A is the set to be sorted, *attr* is the attribute of the set to be acted on, and *order* is the order. $\text{Select}(A, \text{attr}, \text{range})$ defines the *Select* operator, where *range* is the selected range.

3.3.2. Constraint Detection

When the model receives the question S_5 , it is preprocessed and delivered to the semantic encoding layer, which uses Bert as the pretrained model to generate the matrixed representation $\mathbf{E}_{CD}$ of the question. Following that, $\mathbf{E}_{CD}$ is input into the semantic reasoning layer, which employs a bidirectional recurrent neural network BiGRU to capture the contextual information of the question's words and improve the model's reasoning abilities. The question's forward encoding $\vec{\mathbf{E}}_{CD} = [\vec{\mathbf{e}}_1, \vec{\mathbf{e}}_2, \dots, \vec{\mathbf{e}}_n]$ and backward encoding $\overleftarrow{\mathbf{E}}_{CD} = [\overleftarrow{\mathbf{e}}_1, \overleftarrow{\mathbf{e}}_2, \dots, \overleftarrow{\mathbf{e}}_n]$ are then obtained, and the two are concatenated to generate $\overleftrightarrow{\mathbf{E}}_{CD}$, which is added to the original input $\mathbf{E}_{CD}$ to fuse the information contexts. Finally, this layer produces the encoded information for the question $\mathbf{H}_W \in \mathbb{R}^{n \times q}$, where q is the word vector dimension. Equations (10) and (11) are used in the process:

$$\overleftrightarrow{\mathbf{E}}_{CD} = \text{concat}(\vec{\mathbf{E}}_{CD}, \overleftarrow{\mathbf{E}}_{CD}) \quad (10)$$

$$\mathbf{H}_W = \overleftrightarrow{\mathbf{E}}_{CD} + \mathbf{E}_{CD} \quad (11)$$

$\mathbf{H}_W$ is then sent into the constraint detection layer, which we model as a sequence tagging task, tagging each token with entity tags using TLNO tag (**T** is a temporal constraint, **L** is a spatial constraint, **N** is a numerical constraint, and **O** is other). The constraint detection layer first employs a trainable matrix $\mathbf{W}_D \in \mathbb{R}^{q \times m_t}$ to reduce the dimensionality of $\mathbf{H}_W$, where m_t is the number of tag types, here $m_t = 4$, and then employs softmax as a classifier to predict the corresponding tag $\mathbf{P}_D \in \mathbb{R}^{n \times m_t}$ for each token, implying the probability of the label corresponding to each token predicted by the classifier, for which the loss function is calculated using cross entropy, as shown in Equation (12):

$$\text{Loss}_3 = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^{m_t} y_{i,j} \log p_{i,j} \quad (12)$$

where $y_{i,j} \in \{0, 1\}$, if the i th token's real tag is j then $y_{i,j} = 1$, otherwise $y_{i,j} = 0$. $p_{i,j}$ is the probability of the i th token belongs to the j th tag.

The model's input is a simple question $S_E = \{w_1, w_2, \dots, w_n\}$ and the topic entity e_t . To eliminate information interference and limit the likelihood of ambiguity during path selection, the placeholder $\langle e \rangle$ has been used in place of the topic entity and constraint information in S_E . To decrease the search space and increase path recall, the model employs beam search rather than typical breadth-first search for path selection, and a pseudo-siamese network based on the pre-trained model is used to calculate the similarity between questions and paths. Algorithm 1 gives the execution flow of the HBHR model.

Algorithm 1 the process of HBHR

Input: Knowledge Graph $\mathcal{G}$, Topic Entity e_t , Simple Question S_E

Output: Optimal Path $\mathcal{P}$

```

1: initialization Max Heap  $B$  with Size  $K$ 
2: initialization Similarity Calculate Function  $Sim$  with random weights
3: initialization new Path Set  $D = \{\}$ 
4:  $\mathbf{e}^q \leftarrow questionEncoder(S_E)$  //question embedding
5: time step  $t \leftarrow 0$ 
6: while  $True$  do
7:   if  $t == 0$  then
8:      $D = selectNextPath(e_t)$  //select all path that start from node  $e_t$ 
9:     for  $\mathcal{P} \in D$  do
10:       $\mathbf{e}^p = pathEncoder(\mathcal{P})$  //path embedding
11:       $score = Sim(\mathbf{e}^q, \mathbf{e}^p)$ 
12:       $B_t.push(\{\mathcal{P}, score\})$ 
13:     end for
14:   else
15:     for  $\mathcal{P} \in B_{t-1}$  do //browse all path in max heap  $B_{t-1}$ 
16:        $D = selectNextPath(\mathcal{P}.endnode)$ 
17:       for  $\mathcal{P} \in D$  do
18:         $\mathbf{e}^p = pathEncoder(\mathcal{P})$ 
19:         $score = Sim(\mathbf{e}^q, \mathbf{e}^p)$ 
20:         $B_t.push(\{\mathcal{P}, score\})$ 
21:       end for
22:     end for
23:     if  $B_{t-1} == B_t$  then
24:       break //beam search end
25:     end if
26:   end if
27:    $t++$ 
28:    $D = \{\}$ 
29: end while
30: return  $B_t.top()$ 

```

After receiving S_E , the model vectorizes it in order to obtain $\mathbf{e}^q$. This step prepares the question and candidate paths for the next step of similarity calculation. We use the similarity computation with the question and the candidate paths.

3.4.1. Question Embedding and Path Embedding

The model's question embedding module initially sends the question S_E to the Bert encoder for word-level encoding in order to acquire the word matrix $\mathbf{E}_B = [\mathbf{e}_1^B, \mathbf{e}_2^B, \dots, \mathbf{e}_n^B]$. Then $\mathbf{E}_B$ is transmitted to FFN, where the fully connected layer is configured for two purposes: one, to minimize the dimensionality of $\mathbf{E}_B$ for following operations, and the other, to enable the question encoding module to learn. After the fully connected layer, $\mathbf{E}_B$ yields the question matrix $\mathbf{E}_F \in \mathbb{R}^{n \times m_f}$, where m_f is the dimensionality of a single vector. Finally, a pooling layer is used to reduce the dimension of $\mathbf{E}_F$ to obtain the question

vector $\mathbf{e}^q \in \mathbb{R}^b$, so that the output of the question embedding module is of fixed dimension. The pooling method is chosen as mean-pooling, as shown in Equation (14):

$$\mathbf{e}_q = \text{mean pooling}(\mathbf{E}_F) \quad (14)$$

The path embedding module follows the same structure as the question embedding module in that both use Bert as an encoder to obtain the word matrix, then FFN to reduce the dimensionality while making the path embedding module capable of learning, and finally mean-pooling to reduce the dimensionality of the path matrix so that it is similar to the question vector $\mathbf{e}^q$ for similarity calculation. Because the path information is represented by a set of triples, it must be connected to form a sentence based on the same triple nodes before being fed into the model, and the sentence is then sent to the path embedding module. Although the network structure of the question embedding and path embedding modules is identical, the parameters are not.

3.4.2. Path Selection and Similarity Calculation

For reasoning, the HBHR model employs beam search rather than typical breadth-first search, which reduces the search space while raising the hit rate of accurate answers. The model internally builds a list B of capacity K , denoted by *Top-K List* in Figure 5, for recording the K paths that are currently most similar to the question, and then sorts the paths in descending order of similarity to the question. The model is initialized with B empty, and the set $D = \{p_1, p_2, \dots, p_n\}$ created by all paths from topic entity e_t with a one-hop distance is chosen, and each path p_i in D is encoded using the path embedding model to obtain the vector representation of the path $\mathbf{e}_i^p$. The similarity between $\mathbf{e}_i^p$ and $\mathbf{e}^q$ is calculated below using the cosine similarity function, as shown in Equation (15):

$$\cos(\mathbf{e}_i^p, \mathbf{e}^q) = \frac{\mathbf{e}_i^p \bullet \mathbf{e}^q}{|\mathbf{e}_i^p| \times |\mathbf{e}^q|} \quad (15)$$

The outcome is then entered into B . If the number of paths in B exceeds K , the path with the least similarity, which is the last in the list, is deleted. After traversing all the paths in D , we can obtain the K paths with the highest similarity to the question starting from e_t .

The model generates a temporal snapshot B_1 during the second round of path selection. At this time, B_1 contains K paths $D = \{p_1, p_2, \dots, p_K\}$ that are most likely to be the correct answer. For each of these paths p_i , we select the next hop of its tail node $p_i.\text{endNode}$ that has not yet been visited as a new node to extend the path to form a new set of paths $D_i = \{p_{i1}, p_{i2}, \dots, p_{iK}\}$. Following that, we continue to encode each path p_{ij} in D_i using the path embedding model to obtain $\mathbf{e}_{ij}^p$, calculate the similarity between $\mathbf{e}_{ij}^p$ and $\mathbf{e}^q$, and place it in list B . If B overflows, the first K paths are still recorded in the order of highest to lowest similarity. When the above operation is performed for all paths in B , i.e., after the 2nd round of path selection is completed, compare the paths in list B with those in B_1 ; if they are different, it means there are new candidate paths generated and continue the 3rd round of path selection; if they are the same, it means there are no new paths generated in this round of path selection, end the path selection, and take the path with the highest similarity as the correct path $\mathcal{P}$.

After the model has generated $\mathcal{P}$ through S_E and e_t , the constraint information needs to be bound to $\mathcal{P}$. The constraint extraction in Section 3.3 begins with obtaining the question's word vector matrix $\mathbf{E}_{CD}$ using Bert, and ends with determining the constrained object O . We generate the vector representation $\mathbf{e}_s$ of O by summing and averaging the word vector matrix $\mathbf{E}_{CD}^O$ representing O from the $\mathbf{E}_{CD}$. $\mathcal{P}$ is represented by a set consisting of $(\text{head}, \text{relation}, \text{tail})$. The path embedding module's initial phase will encode $\mathcal{P}$ using Bert, and all of the $(\text{head}, \text{tail})$ vectors in it will be extracted independently to generate $\mathbf{E}_O = [\mathbf{e}_1^O, \mathbf{e}_2^O, \dots, \mathbf{e}_K^O]$. Next, the similarity between $\mathbf{e}_s$ and each vector in $\mathbf{E}_O$ is calculated

using cosine similarity, and the node with the highest similarity is selected as the bound node for the constraint. This part does not require any training.

4. Experimental Section

The main research task of this paper is complex question-answering tasks over knowledge graphs, for which we use a pipeline model to divide it into three subtasks and optimize each subtask to achieve the best overall task performance. The experiments are divided into three parts: the first tests the complex question decomposition model's ability to decompose complex questions into simple ones; the second tests the question constraint extraction model and the question reasoning model's ability to reason about simple questions; and the third tests the overall ability of the framework proposed in this paper to deal with complex questions.

First, the basic experiments of each part are carried out to compare the model proposed in this paper with other baseline models, and to quantify the performance capability of different models under different datasets using relevant evaluation metrics. Since the HBHR model utilizes beam search in the second part, we then test the impact of the maximum number of paths K set within the list on the functionality of the HBHR model in beam search. Finally, we perform ablation experiments to examine how dependency encoder and transfer learning affect the BTAM model, as well as how question constraint extraction affect the model as a whole.

4.1. Datasets

The experiments in this paper were carried out on five separate baseline datasets, with different datasets used for each part of the experiment. Table 2 displays the information and applications of these datasets.

Table 2. Experimental dataset.

Name	Topic Entity Type	Relation Type	Number of Questions	Apply Section
ComplexWebQuestions	Multi-topic entities	Multi-relations	34,672	Part I,III
SimpleQuestions	Single-topic entity	Single relation	108,422	Part I
LC-QuAD2.0	Single-topic entity	Multi-relations	30,224	Part I
webQuestionsSP	Single-topic entity	Multi-relations	4737	Part II
complexQuestions	Single-topic entity	Multi-relations	2100	Part II

ComplexWebQuestions (CWQS) [20] dataset is constructed based on WebQuestionsSP (WQSP) [38] dataset, where the questions in WQSP dataset all contain only one topic entity. Simple questions are generated by developing rules to ask a keyword of a question in WQSP dataset, and the original keyword is replaced by that simple question to form complex questions in CWQS dataset, and then the question is manually rewritten to ensure that the question matches natural language. This ensures that the questions in CWQS dataset are multi-topic entities multi-relations questions.

SimpleQuestions [11] dataset and LC-QuAD [39] dataset are used for model transfer training. The question decomposition model's core function is to find single-topic entity questions from complex questions. To generate the transfer training dataset, a total of 80,000 single-topic entities questions were extracted from SimpleQuestions and LC-QuAD datasets in the first stage of the experiment, and the number of random noise words were added to each question. The model's ability to recognize single-topic entities is improved during the training process by allowing the model to recognize the original question in questions with noisy words.

WQSP dataset and complexQuestions (CQ) dataset are used as the experimental dataset for the second part of the experiments, which involves testing the performance of the constraint extraction and question reasoning models. Unlike other complex questions datasets, CQ dataset includes constraint information for each question. The reason for

using these datasets is that they contain questions with only one topic entity and multiple relations, and most of the questions with constraint information.

For the third part of the experiment, i.e., the joint testing of question decomposition, constraint extraction and question reasoning, CWQS is used as the experimental dataset, which is a multi-topic entity multi-relations question-answering dataset with constraints.

4.2. Evaluation Metrics

We evaluate each model using three metrics: precision P , recall R , and $F1$ value, where the $F1$ value needs to be calculated based on P and R . The calculation formulas are shown in Equations (16), (17) and (18), respectively:

$$P = \frac{TP}{TP + FP} \quad (16)$$

$$R = \frac{TP}{TP + FN} \quad (17)$$

$$F1 = \frac{2 \times P \times R}{P + R} \quad (18)$$

The preceding equations are also understood in conjunction with the confusion matrix, as stated in Table 3.

Table 3. Confusion matrix.

	Real Positive Value	Real Negative Value
Predicted Positive Value	TP	FP
Predicted Negative Value	FN	TN

In the above table, P/N denotes whether the model prediction classification is true or false, and T/F denotes if the model prediction results match or do not match the true results. TP denotes the number of samples correctly classified as positive, TN denotes the number of samples correctly classed as negative, FP denotes the number of samples misclassified from negative to positive, and FN denotes the number of samples misclassified from positive to negative. The accuracy rate is utilized as the evaluation metric because this is a classification model. The accuracy rate formula is provided in Equation (19). Let β be the total number of data and η be the number of valid classifications.

$$Acc = \frac{\eta}{\beta} \times 100\% \quad (19)$$

4.3. Baseline Methods

In the first part of the experiment, we choose the following baseline models to compare with our model.

- SPLITQA [20]: This model decomposes complex questions into a set of simple questions using a pointer network. It builds the encoder and decoder of the pointer network using a one-layer BiGRU network. First, it vectorizes the words using GloVe. Next, it creates a simple question sequence by copying the words from the original input question. Finally, it uses cross-entropy for training.
- DECOMPRC [22]: The pre-trained model Bert is used as the encoder in the question decomposition part of this model, and simply a linear layer is utilized in the decoding phase, ultimately the softmax mechanism is used to label the simple question locations from the input sequence.
- BiLSTM_CRF [40]: BiLSTM is used as the encoder and conditional random field (CRF) is used as the decoder to find the optimal path.

- BiLSTM_CNN_CRF [41]: This model uses BiLSTM as an encoder, CNN as an reasoning layer, and CRF as a decoder to find the best path.
- Bert_BiLSTM_CRF: This model employs Bert as the encoder, followed by BiLSTM as the intermediate layer for reasoning and CRF as the decoder to determine the optimum path.

In the second and third part of the experiments, we selected the following baseline models.

- IRN [8]: This model is an interpretable hop-by-hop reasoning network that reasons the entire path from the topic entity in the question to each candidate response node in the subgraph. The model uses a bag-of-words model for encoding and hence has a loss of semantic representation of the question.
- SRN [35]: This is a reinforcement learning-based hop-by-hop reasoning model that includes a perceptron to handle question encoding during each hop. However, it limits the model's scalability.
- UHop [28]: This model is built on information extraction and focuses on solving multi-hop questions. It is a multi-hop relation detection framework that detects numerous relation structures but does not handle constraint information individually.
- PullNet [29]: This model uses a fusion idea to fuse data from other knowledge sources to assist in knowledge reasoning early in the model.
- TransferNet [42]: This model uses entity recognition, entity linking, and relation extraction to generate text-formed triples, which are then used to supplement the incomplete KG, with answers inferred by transferring entity scores along relation scores.
- MRP-QA [43]: This model can use information across multiple reasoning paths and simply needs labeled answers as supervision. To train the model, a marginalization probability objective function is used.
- HTL [44]: This model categorizes natural language questions into answer templates. To aid a Tree-LSTM in identifying the most significant information, an attention mechanism is built.

4.4. Experimental Setup

Our computational hardware environment includes a Quadro RTX 8000 GPU and 256 GB of RAM. Python is used as the development language, and Pytorch 1.12 is used as the framework.

The experimental dataset was separated into a training set, a test set, and a validation set in the ratio of 8:1:1 in the first part of the experiments. The transfer training dataset is separated into training and test sets in an 8:2 ratio. The dataset is divided into training, validation, and test sets in a 4:1:1 ratio for all data in the second and third parts of the experiments.

4.5. Overall Results on Part I Experiment

We first calculated the P , R , and $F1$ values of the complex question decomposition model BTAM with other baseline models on CWQS dataset, and the results are shown in Table 4. It can be seen from Table 4 that the three evaluation metrics of BTAM model we proposed on CWQS dataset outperformed the baseline models with P , R , and $F1$ values of 89.4%, 78.7%, and 83.7%, respectively.

Table 4. Performance of different models on ComplexWebQuestions dataset.

Model	P	R	$F1$
SPLITQA	57.6%	43.0%	49.2%
DECOMPRC	56.3%	48.0%	51.8%
BiLSTM_CRF	48.2%	38.0%	42.5%
BiLSTM_CNN_CRF	55.4%	38.2%	45.2%
Bert_BiLSTM_CRF	83.5%	65.6%	73.5%
BTAM	89.4%	78.7%	83.7%

SPLITQA model performs poorly on CWQS dataset test, with its $F1$ value 34.5% lower than BTAM model. This is due to SPLITQA model's relatively simplistic structure, which makes the model less capable of extracting semantic features from questions when simple questions are given as input.

DECOMPRC model also performs poorly since it only examines the semantic information of the questions and lacks context when only ten to twenty words are provided as input to the questions.

The $F1$ values for the BiLSTM_CRF and BiLSTM_CNN_CRF models are 42.5% and 45.2%, respectively. This is due to the tiny amount of data in the dataset and the lack of other contextual information, which makes learning the true distribution of the problem challenging for the small-scale model. Because of the use of pre-training for encoding, the Bert_BiLSTM_CRF model outperforms the BiLSTM_CRF without pre-training by 31% in $F1$.

The BTAM model we proposed employs a dependency encoder for question feature extraction, allowing the model to extract semantic information from the question even when contextual information is unavailable. Furthermore, this model uses transfer training, which improves understanding of the questions.

4.6. Overall Results on Part II Experiment

4.6.1. Basic Experiment

In this section, the constraint extraction model and the question reasoning model are evaluated as a whole and compared to other baseline models on WQSP dataset and CQ dataset, respectively, with the findings presented in Figure 6. The figure shows that the $F1$ values of the proposed model outperform the baseline models on both datasets, by 74.5% and 77.8%, respectively.

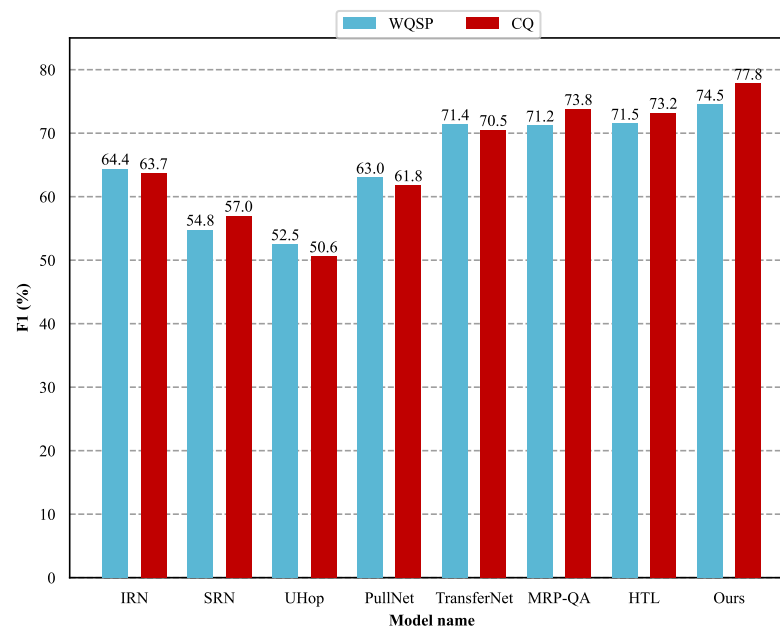


Figure 6. Performance of each model on the test set.

SRN is a weakly supervised model that ignores the relations' overall semantic information. Both IRN and UHop are relational supervised models that neglect the local information of the relations, and both use a delayed termination mechanism that does not adequately filter the information during the hop-by-hop updating of the questions. PullNet and TransferNet are designed to be able to use both documents and knowledge bases as knowledge sources, and because only the knowledge graph is employed as the information source in this experiment, the effect would be affected. For MRP-QA, although it uses multi-paths reasoning, when the number of hops increases and the constraint relations become complex, it instead performs worse than our proposed model due to the increased

choice of paths. Because the knowledge base schema was not incorporated as inputs during the Hereditary Tree-LSTM training, HTL is difficult to identify between distinct multi-hops.

Because of the independent treatment of the constraint information in the question, our proposed model performs better in handling complex questions with constraints. Furthermore, the model employs a question reasoning system based on beam search, which increases search efficiency while decreasing search space.

4.6.2. Beam Search

Because HBHR model employs beam search, the choice of hyperparameter K , which regulates the width in beam search, has a significant impact on the model's performance. We created an experiment to study the effect of K on the model in the range of 1 to 7. Figure 7 depicts the outcomes of tests performed on the WQSP dataset.

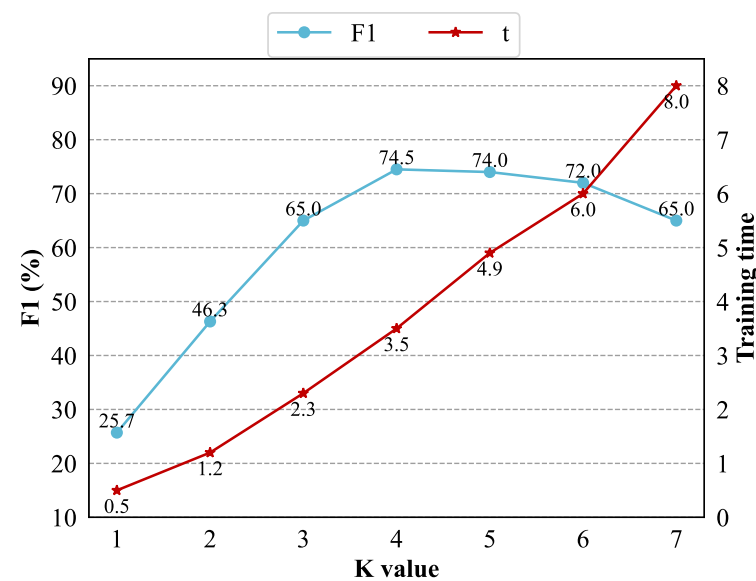


Figure 7. Effects of different K values on model performance.

Figure 7 demonstrates that the model's performance increases and subsequently decreases with the increase of K value, with the $F1$ value being the highest when K is 4. This is because when K is 1, the model can only keep the current optimal path and loses the ability to explore potential optimal paths, making it easy to find suboptimal solutions, whereas when K is too large, an excessive number of paths are kept at the end of each cycle, and many of these paths are suboptimal or incorrect solutions, causing significant interference to the search algorithm. The figure also shows that the training time increases rapidly as K increases.

4.7. Overall Results on Part III Experiment

To test the capability of our proposed framework for handling multi-topic entity multi-relation questions with constraints, the whole model is tested on CWQS dataset, and the results are compared with the baseline models, as shown in Figure 8.

As seen in Figure 8, the model we proposed outperforms other types of models on complicated questions involving a multi-topic entity with multi-relation and constraints. This is primarily due to our model's question decomposition strategy, which gradually decomposes the multi-topic entity multi-relation question with constraints into a single-topic entity question without constraints before performing question reasoning, constraint processing, and logical combination for each single-topic entity. By using this simplification approach, the proposed model performs well in this task. It should be noted that PullNet model is designed to integrate multiple knowledge sources, hence performance suffers when only knowledge graphs are employed.

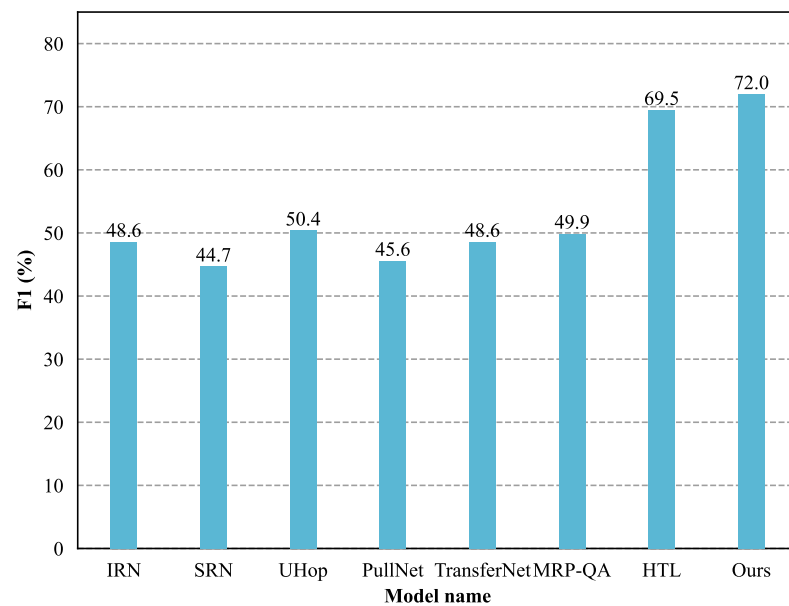


Figure 8. Performance comparison of the model on CWQS dataset.

4.8. Ablation Studies

4.8.1. Dependency Encoder

To demonstrate the importance of the dependency encoder in BTAM model, we separately tested the model's performance with and without the dependency encoder on CWQS dataset, and the results are given in Figure 9.

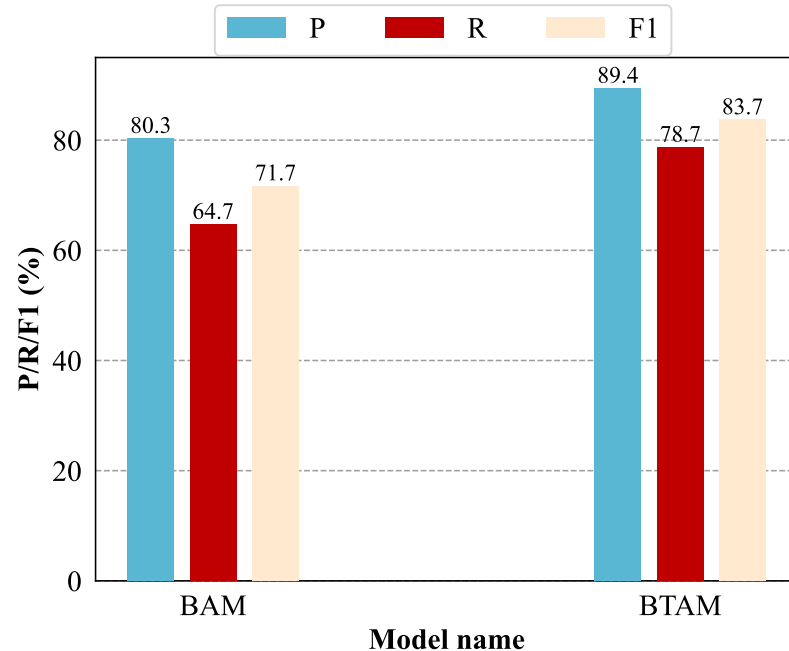


Figure 9. Dependency encoder ablation experimental results.

In Figure 9, BAM represents the model without the dependency encoder, while BTAM represents the model with the dependency encoder. The figure shows that when the dependency encoder is not used, the *F1* value drops by 12.0%. This is because the dependency encoder helps the semantic encoder extract information from the question phrases, resulting in a more complete vector representation of the question sentence than the semantic encoder alone, enhancing the model's performance.

4.8.2. Transfer Learning

To examine the influence of transfer learning on BTAM model, an experiment is created to test the model directly after training on CWQS dataset without utilizing transfer learning. Figure 10 shows a comparison of the results with the model using transfer learning.

As shown in Figure 10, the model improves its F1 value by 11.4% when using transfer learning versus not using transfer learning, which is a more significant improvement. The model that used transfer learning was initially trained on the transfer learning dataset before being trained on the experimental dataset, allowing the model to gain preliminary knowledge of simple questions. The model that does not use transfer learning can only be trained on the experimental dataset, which contains nearly 70% less training data than the former, so the results are lower.

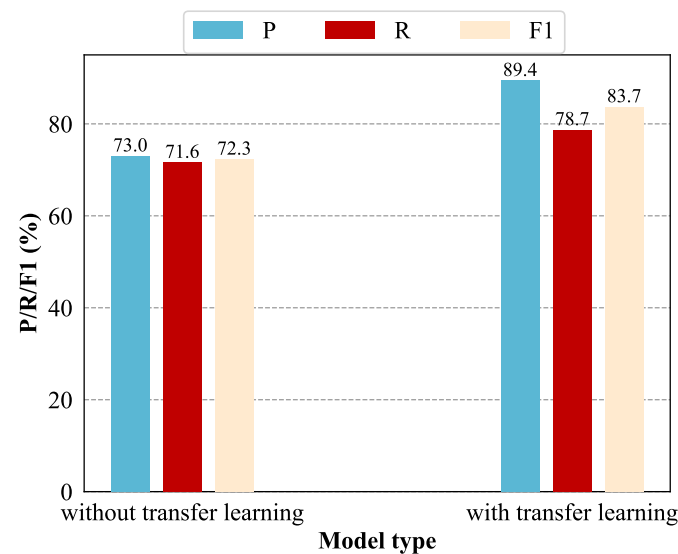


Figure 10. The impact of transfer learning on the model performance.

4.8.3. Question Constraint Extraction

To investigate the impact of the constraint extraction model on HBHR model, we create an experiment that is answered directly using the question reasoning model without going through the constraint extraction step, and the performance on the experimental dataset is shown in Figure 11.

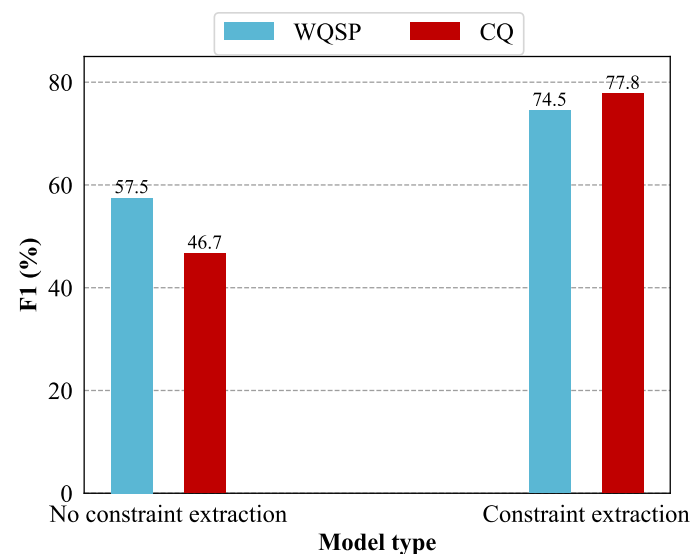


Figure 11. The impact of the constraint extraction model on HBHR model.

From Figure 11, it can be seen that not processing the constraint information in the question can lead to a sudden drop in the effectiveness of the question reasoning model. This is because if the constraint information in the question is not processed, the reasoning mechanism is unable to handle the redundant information, which are noisy data for the question reasoning model, and they not only do not improve the accuracy, but also interfere with the normal operation of the reasoning model, resulting in performance degradation.

5. Conclusions

In this paper, we propose a complex question-answering framework over knowledge graphs that applies a deep learning approach to the complex question-answering domain. The model divides the problem into three subtasks based on the process of humans answering complex questions: question decomposition, constraint extraction, and question reasoning. Extensive experimental and ablation studies show that our proposed model outperforms previous baseline models for complex question-answering tasks over knowledge graphs. To improve outcomes, we plan to add question combination logic and question classification to the model in future work.

There are still some challenges and limitations to overcome. The small size of the training dataset used in this model is not necessarily generalizable to large-scale datasets, which may limit the accuracy of the model. In addition, all datasets in this experiment are in English, and when the dataset is in other languages, such as Chinese, it may be challenging for the model to decompose the questions and extract the constraints due to grammatical differences. Finally, research on the model's interpretability should be improved. One example is the modification of the word dependency matrix at multiple hops.

The following outlines our planned research initiatives for the future. (1) We intend to extend our proposed model by using larger datasets with more complex problems. (2) Due to the different sentence structures caused by language differences, we intend to use multilingual datasets to train the model so that it can be better adapted to different languages. (3) We will increase the study of model interpretability to improve the scalability of the model.

Author Contributions: Conceptualization, L.Y. and Y.D.; methodology, L.Y.; software, H.G.; validation, Y.D.; formal analysis, H.G.; investigation, W.C.; resources, Y.D.; data curation, W.C.; writing—original draft preparation, H.G.; writing—review and editing, L.Y., Y.D. and W.C.; visualization, W.C.; supervision, Y.D.; project administration, L.Y.; funding acquisition, L.Y. All authors have read and agreed to the published version of the manuscript.

Funding: Supported by the National Key Research and Development Program of China (No. 2021YFF0901200).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The model is trained on the ComplexWebQuestions (CWQS), SimpleQuestions, LC-QuAD, webQuestionsSP (WQSP) and complexQuestions (CQ) datasets, respectively. CWQS is used as a dataset for multi-topic entities with multi-relations questions. Dataset link: <https://www.dropbox.com/sh/7pkwkrfnwqhsnp0/AACuu4v3YNkhirzBOeeaHYala>. SimpleQuestions and LC-QuAD is used as the transfer training dataset for this model. Dataset link: <https://research.facebook.com/downloads/babi/>, <https://research.facebook.com/downloads/babi/>; <http://lc-quad.sda.tech/>. WQSP is a single topic entity with multi-relations questions dataset which is used to train the model for constraint extraction. Dataset link: <https://worksheets.codalab.org/worksheets/0xba659fe363cb46e7a505c5b6a774dc8a>, <https://worksheets.codalab.org/worksheets/>. CQ is a dataset built specifically for complex questions and is used to train the model's reasoning ability. Dataset link: <https://github.com/JunweiBao/MulCQA/tree/ComplexQuestions>.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Fensel, D.; Şimşek, U.; Angele, K.; Huaman, E.; Kärle, E.; Panasiuk, O.; Toma, I.; Umbrich, J.; Wahler, A. Introduction: What is a knowledge graph? In *Knowledge Graphs: Methodology, Tools and Selected Use Cases*; Springer: Cham, Switzerland, 2020; pp. 1–10.
2. Vrandečić, D.; Krötzsch, M. Wikidata: A free collaborative knowledgebase. *Commun. ACM* **2014**, *57*, 78–85. [\[CrossRef\]](#)
3. Auer, S.; Bizer, C.; Kobilarov, G.; Lehmann, J.; Cyganiak, R.; Ives, Z. DBpedia: A Nucleus for a Web of Open Data. *Lect. Notes Comput. Sci.* **2007**, *6*, 722–735. [\[CrossRef\]](#)
4. Bollacker, K.; Evans, C.; Paritosh, P.; Sturge, T.; Taylor, J. Freebase: A collaboratively created graph database for structuring human knowledge. In Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, Vancouver, BC, Canada, 10–12 June 2008; pp. 1247–1250.
5. Bi, X.; Nie, H.; Zhang, X.; Zhao, X.; Yuan, Y.; Wang, G. Unrestricted multi-hop reasoning network for interpretable question answering over knowledge graph. *Knowl.-Based Syst.* **2022**, *243*, 108515. [\[CrossRef\]](#)
6. Yih, S.W.t.; Chang, M.W.; He, X.; Gao, J. Semantic parsing via staged query graph generation: Question answering with knowledge base. In Proceedings of the Joint Conference of the 53rd Annual Meeting of the ACL and the 7th International Joint Conference on Natural Language Processing of the AFNLP, Beijing, China, 2–7 August 2015.
7. Yu, M.; Yin, W.; Hasan, K.S.; Santos, C.d.; Xiang, B.; Zhou, B. Improved neural relation detection for knowledge base question answering. *arXiv* **2017**, arXiv:1704.06194.
8. Zhou, M.; Huang, M.; Zhu, X. An interpretable reasoning network for multi-relation question answering. *arXiv* **2018**, arXiv:1801.04726.
9. Lan, Y.; Jiang, J. Query Graph Generation for Answering Multi-hop Complex Questions from Knowledge Bases. In Proceedings of the Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 969–974. [\[CrossRef\]](#)
10. Petrochuk, M.; Zettlemoyer, L. Simplequestions nearly solved: A new upperbound and baseline approach. *arXiv* **2018**, arXiv:1804.08798.
11. Bordes, A.; Usunier, N.; Chopra, S.; Weston, J. Large-scale simple question answering with memory networks. *arXiv* **2015**, arXiv:1506.02075.
12. Azeem, M.; Jamil, M.K.; Shang, Y. Notes on the localization of generalized hexagonal cellular networks. *Mathematics* **2023**, *11*, 844. [\[CrossRef\]](#)
13. Perez, J.; Arenas, M.; Gutierrez, C. Semantics and complexity of SPARQL. In Proceedings of the 5th International Conference on The Semantic Web, Athens, GA, USA, 5–9 November 2006; Volume 34. [\[CrossRef\]](#)
14. Hartig, O.; Pérez, J. Semantics and Complexity of GraphQL. In Proceedings of the WWW '18: Proceedings of the 2018 World Wide Web Conference, Lyon, France, 23–27 April 2018; pp. 1155–1164. [\[CrossRef\]](#)
15. Bao, J.; Duan, N.; Yan, Z.; Zhou, M.; Zhao, T. Constraint-based question answering with knowledge graph. In Proceedings of the 26th International Conference on Computational Linguistics: Technical Papers (COLING 2016), Osaka, Japan, 11–16 December 2016; pp. 2503–2514.
16. Qin, K.; Li, C.; Pavlu, V.; Aslam, J. Improving query graph generation for complex question answering over knowledge base. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Online, 7–11 November 2021; pp. 4201–4207.
17. Luo, K.; Lin, F.; Luo, X.; Zhu, K. Knowledge base question answering via encoding of complex query graphs. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 31 October–4 November 2018; pp. 2185–2194.
18. Maheshwari, G.; Trivedi, P.; Lukovnikov, D.; Chakraborty, N.; Fischer, A.; Lehmann, J. Learning to rank query graphs for complex question answering over knowledge graphs. In Proceedings of the International Semantic Web Conference, Auckland, New Zealand, 26–30 October 2019; pp. 487–504.
19. Zhu, S.; Cheng, X.; Su, S. Knowledge-based question answering by tree-to-sequence learning. *Neurocomputing* **2020**, *372*, 64–72. [\[CrossRef\]](#)
20. Talmor, A.; Berant, J. The web as a knowledge-base for answering complex questions. *arXiv* **2018**, arXiv:1803.06643.
21. Kalyanpur, A.; Patwardhan, S.; Boguraev, B.; Lally, A.; Chu-Carroll, J. Fact-based question decomposition in DeepQA. *IBM J. Res. Dev.* **2012**, *56*, 13:1–13:11. [\[CrossRef\]](#)
22. Min, S.; Zhong, V.; Zettlemoyer, L.; Hajishirzi, H. Multi-hop reading comprehension through question decomposition and rescoring. *arXiv* **2019**, arXiv:1906.02916.
23. Yang, H.; Wang, H.; Guo, S.; Zhang, W.; Chen, H. Learning to decompose compound questions with reinforcement learning. In Proceedings of the International Conference on Learning Representations (ICLR), New Orleans, LA, USA, 6–9 May 2019.
24. Pan, L.; Chen, W.; Xiong, W.; Kan, M.Y.; Wang, W.Y. Unsupervised multi-hop question answering by question generation. *arXiv* **2020**, arXiv:2010.12623.
25. Perez, E.; Lewis, P.; Yih, W.t.; Cho, K.; Kiela, D. Unsupervised question decomposition for question answering. *arXiv* **2020**, arXiv:2002.09758.
26. Xie, K.; Wiegrefe, S.; Riedl, M. Calibrating Trust of Multi-Hop Question Answering Systems with Decompositional Probes. *arXiv* **2022**, arXiv:2204.07693.

27. Chen, Z.; Zhao, X.; Liao, J.; Li, X.; Kanoulas, E. Temporal knowledge graph question answering via subgraph reasoning. *Knowl.-Based Syst.* **2022**, *251*, 109134. [\[CrossRef\]](#)
28. Chen, Z.Y.; Chang, C.H.; Chen, Y.P.; Nayak, J.; Ku, L.W. UHop: An unrestricted-hop relation extraction framework for knowledge-based question answering. *arXiv* **2019**, arXiv:1904.01246.
29. Sun, H.; Dhingra, B.; Zaheer, M.; Mazaitis, K.; Salakhutdinov, R.; Cohen, W.W. Open domain question answering using early fusion of knowledge bases and text. *arXiv* **2018**, arXiv:1809.00782.
30. Sun, H.; Bedrax-Weiss, T.; Cohen, W.W. Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text. *arXiv* **2019**, arXiv:1904.09537.
31. Yang, M.C.; Duan, N.; Zhou, M.; Rim, H.C. Joint relational embeddings for knowledge-based question answering. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, 25–29 October 2014; pp. 645–650.
32. Xiong, W.; Yu, M.; Chang, S.; Guo, X.; Wang, W.Y. Improving question answering over incomplete kbs with knowledge-aware reader. *arXiv* **2019**, arXiv:1905.07098.
33. Yang, Z.; Ye, J.; Wang, L.; Lin, X.; He, L. Inferring substitutable and complementary products with Knowledge-Aware Path Reasoning based on dynamic policy network. *Knowl.-Based Syst.* **2022**, *235*, 107579. [\[CrossRef\]](#)
34. Xiong, W.; Hoang, T.; Wang, W.Y. Deeppath: A reinforcement learning method for knowledge graph reasoning. *arXiv* **2017**, arXiv:1707.06690.
35. Qiu, Y.; Wang, Y.; Jin, X.; Zhang, K. Stepwise reasoning for multi-relation question answering over knowledge graph with weak supervision. In Proceedings of the 13th International Conference on Web Search and Data Mining, Houston, TX, USA, 3–7 February 2020; pp. 474–482.
36. Liu, H.; Zhou, S.; Chen, C.; Gao, T.; Xu, J.; Shu, M. Dynamic knowledge graph reasoning based on deep reinforcement learning. *Knowl.-Based Syst.* **2022**, *241*, 108235. [\[CrossRef\]](#)
37. Manning, C.D.; Surdeanu, M.; Bauer, J.; Finkel, J.R.; Bethard, S.; McClosky, D. The Stanford CoreNLP natural language processing toolkit. In Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, Baltimore, MD, USA, 22–27 June 2014; pp. 55–60.
38. Yih, W.t.; Richardson, M.; Meek, C.; Chang, M.W.; Suh, J. The value of semantic parse labeling for knowledge base question answering. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Short Papers), Berlin, Germany, 7–12 August 2016; Volume 2, pp. 201–206.
39. Trivedi, P.; Maheshwari, G.; Dubey, M.; Lehmann, J. Lc-quad: A corpus for complex question answering over knowledge graphs. In Proceedings of the International Semantic Web Conference, Vienna, Austria, 21–25 October 2017; pp. 210–218.
40. Lample, G.; Ballesteros, M.; Subramanian, S.; Kawakami, K.; Dyer, C. Neural architectures for named entity recognition. *arXiv* **2016**, arXiv:1603.01360.
41. Ma, X.; Hovy, E. End-to-end sequence labeling via bi-directional lstm-cnns-crf. *arXiv* **2016**, arXiv:1603.01354.
42. Shi, J.; Cao, S.; Hou, L.; Li, J.; Zhang, H. Transfernet: An effective and transparent framework for multi-hop question answering over relation graph. *arXiv* **2021**, arXiv:2104.07302.
43. Wang, Y.; Jin, H. A New Concept of Knowledge based Question Answering (KBQA) System for Multi-hop Reasoning. In Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Online, 10–15 July 2022; pp. 4007–4017.
44. Gomes Jr, J.; de Mello, R.C.; Ströele, V.; de Souza, J.F. A hereditary attentive template-based approach for complex Knowledge Base Question Answering systems. *Expert Syst. Appl.* **2022**, *205*, 117725. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.